

NYCU-EE IC LAB – Fall 2026

Lab03 Exercise

Design: Zuma Game

Data Preparation

1. Extract files from TA's directory:

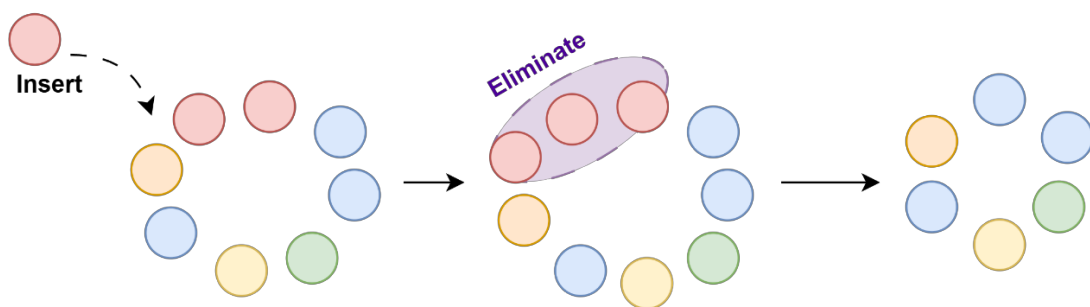
```
% tar xvf ~iclabTA01/Lab03_2026F.tar
```

2. The extracted LAB directory contains:

- a. **00_TESTBED**
- b. **01_RTL**
- c. **02_SYN**
- d. **03_GATE**
- e. **09_SUBMIT**

Design Description

ZUMA is a ring-based bead elimination game. The beads are arranged in a circular sequence, and each bead has one of eight colors. Whenever the Host system shoots a new bead into the ring, the new bead is inserted at the specified logical position. If three or more consecutive beads of the same color are connected, the entire same-color segment is eliminated. The removal may cause the two sides of the eliminated segment to become adjacent and trigger another elimination. This process continues until no further elimination can occur.



In this lab, your task is to implement the ZUMA operation in `ZUMA.v` and create a verification pattern in `PATTERN.v`. Your design should maintain the logical order of the circular ring, process each shot, perform all cascading eliminations, and report the elimination result of every cascade level. Your pattern should integrate stimulus generation, environment checking, and response checking.

- **ZUMA.v**

Whenever the Host system inputs a shot, ZUMA will insert the new bead into the current ring and update the ring according to the cascading elimination rules. For each shot, the design must output

the total number of cascade levels and the eliminated color and bead count of each level. If no elimination occurs, the design must still generate one valid output cycle with `chain_num = 0`.

Logical Definition of the Ring

At any time, the beads in the ring have a logical clockwise order with logical index $0 \sim M-1$, where M is the current number of beads in the ring.

Event	Logical Index Rule
Initial loading	The first loaded bead becomes logical index 0; the following beads are assigned increasing indices in loading order.
Insertion	A new bead inserted after logical index k becomes the new logical index $k+1$. All following beads shift by one logical index.
Elimination	If the bead at logical index 0 survives, it remains logical index 0. If logical index 0 is eliminated, the first surviving bead immediately clockwise after the eliminated segment becomes the new logical index 0.
Empty ring	No logical index exists. For the next shot, shot_pos is ignored and the new bead becomes logical index 0.

Since the structure is a ring, logical index $M-1$ is adjacent to logical index 0. Therefore, every same-color segment check must consider wrap-around across the end and the beginning of the logical sequence.

Ring Loading Phase

Before shooting begins, the initial ring is loaded through **in_valid** and **in_color**.

1. **in_valid** will stay high for exactly **ring_len** consecutive cycles.
2. One initial bead is delivered in each cycle through **in_color**.
3. **ring_len** is valid only in the **first** cycle of the loading phase.
4. **out_valid** must remain low during the entire loading phase.
5. When **in_valid** is asserted again after a game has finished, it represents the beginning of a new pattern/game. The design must **discard all previous ring state and load the new initial ring**.

Shooting and Cascading Elimination

For every valid shot, the following operations are performed in order:

1. Insert **shot_color** after logical index **shot_pos**.
2. Find the maximal consecutive same-color segment that contains the newly inserted bead, including possible wrap-around across logical index $M-1$ and 0.
3. If the segment contains at least 3 beads, eliminate the entire segment. This is cascade level 1.

4. After the elimination, the two surviving sides of the removed segment become adjacent. Check the maximal same-color segment formed at this newly created junction.
5. If the newly joined segment contains at least 3 beads, eliminate the entire segment. This is the next cascade level.
6. Repeat the previous step until the new junction does not form a same-color segment of length 3 or more, or fewer than 3 beads remain in the ring.

For each cascade level, the design reports:

- **elim_color**: the color eliminated at this level.
- **elim_cnt**: the total number of beads eliminated at this level.

The cascade result must be output in the exact order in which the eliminations occur: level 1, level 2, ..., level n.

Important Conditions

1. The initial ring is **guaranteed to contain no same-color segment of length 3 or more**, including wrap-around between the last and first bead.
2. When the ring is empty, **shot_pos** is guaranteed to be **0** and is ignored. The shot bead becomes the only bead in the ring and no elimination occurs.
3. If the ring contains 1 or 2 beads, no elimination can occur unless the current shot makes a same-color segment of at least 3 beads.
4. It is legal for a cascade to eliminate the entire ring. The next shot then starts again from a one-bead ring.
5. An eliminated segment may cross logical index M-1 to logical index 0. Such a wrap-around segment must be handled as one continuous segment.
6. At each cascade level, there is only one elimination segment generated from the unique newly joined boundary. Therefore, the cascade proceeds as one linear sequence of elimination levels.
7. The number of beads in the ring will never exceed **256** in the provided test patterns.

Operational Example

Assume the following color encoding:

R = 1

Y = 2

G = 3

B = 4

The initial ring contains 9 beads:

logical idx: 0 1 2 3 4 5 6 7 8

Shot #1

Therefore:

color: R Y Y G G Y Y B R
shot_color = G, shot_pos = 3
insert: R Y Y [G G] Y Y B R -> level 1: (G, 3)
^
new bead
join: R [Y Y Y Y] B R -> level 2: (Y, 4)
join: R B R -> different colors, stop
chain_num = 2
level 1 = (elim_color, elim_cnt) = (G, 3)
level 2 = (elim_color, elim_cnt) = (Y, 4)
remaining ring = [R, B, R]

Shot #2

shot_color = B, shot_pos = 1
[R, B, R] -> [R, B, B, R]
The same-color segment length is only 2, so no bead is eliminated:

chain_num = 0
elim_color = 0
elim_cnt = 0

● PATTERN.v

Problem Description

In this lab, you need to read the patterns from the file `input.txt`. The format of `input.txt` is outlined below:

```
2 // 2 patterns (games) in this file
9 5 // pattern NO.1: ring_len = 9, shot_num = 5
1 2 2 3 3 2 2 4 1 // initial ring colors, logical index 0 ~ 7
3 3 // shot NO.1: shot_color = 3, shot_pos = 3
1 2 // shot NO.2: shot_color = 1, shot_pos = 2
4 0 // shot NO.3
4 0 // shot NO.4
2 0 // shot NO.5
64 120 // pattern NO.2: ring_len = 64, shot_num = 120
5 1 1 6 2 2 ... // 64 initial colors
```

...

// 120 shots

1. The first number **N** at the beginning of the file is the number of patterns. The rest of the file is divided into **N** pattern groups.
2. The first line of each pattern group contains two integers: **ring_len** and **shot_num**.
3. **ring_len** is the number of beads in the initial ring.
4. **shot_num** is the number of shots in this pattern. **shot_num** is used only by PATTERN.v and is **not** delivered to the design.
5. The next **ring_len** integers are the colors of the initial ring, starting from logical index 0 and continuing clockwise.
6. The following **shot_num** lines each contain one pair of integers: **shot_color shot_pos**.

Please note that you should calculate the golden answer in the PATTERN. The golden answer from external files is NOT allowed.

All the above files are stored in 00_TESTBED.

Inputs and Outputs (ZUMA.v)

- The following are the definition of input signals

Input Signals	Bit Width	Definition
clk	1	Clock
rst_n	1	Asynchronous active-low reset
in_valid	1	High when an initial-ring bead is valid during the ring loading phase.
ring_len	8	The number of beads in the initial ring, ranging from 4 to 128. Valid only in the first cycle when in_valid is high.
in_color	3	Color of an initial-ring bead. The value ranges from 0 to 7. Valid every cycle when in_valid is high.
shot_valid	1	High for 1 cycle when a shot is valid.
shot_color	3	Color of the shot bead. The value ranges from 0 to 7.
shot_pos	8	Insert the shot bead after logical index shot_pos of the current ring. When the ring is non-empty, the value ranges from 0 to M-1 . When the ring is empty, it is guaranteed to be 0 and is ignored.

- The following are the definition of output signals

Output Signals	Bit Width	Definition
out_valid	1	High when the output is valid.
chain_num	7	The number of cascade levels triggered by the current shot. It is 0

		if no elimination occurs.
elim_color	3	The color eliminated at the current cascade level.
elim_cnt	9	The number of beads eliminated at the current cascade level. For an elimination level, the value ranges from 3 to 256.

1. All input signals will change at the clock negative edge.
2. All input signals are synchronized at the negative edge of the clock.
3. **ring_len** will be given in the first cycle of each ring loading phase when **in_valid** is high.
4. For each pattern, **in_valid** will stay high for exactly **ring_len** consecutive cycles, together with **in_color**. **out_valid** must stay low during the entire ring loading phase.
5. The first **shot_valid** will arrive between 1~4 negative edges of the clock after **in_valid** is pulled down.
6. For each shot, **shot_valid** will stay high for 1 cycle, together with **shot_color** and **shot_pos**. After receiving the shot, the design should process the ring and output the result. The next **shot_valid** will arrive between 1~4 negative edges of the clock after the current **out_valid** is pulled down.
7. The next pattern, indicated by the next assertion of **in_valid**, will arrive between 1~4 negative edges of the clock after the current **out_valid** is pulled down.
8. **chain_num** must remain constant during the entire output period of the same shot.
9. If **chain_num** = n and n >= 1, **out_valid** must stay high for exactly n continuous cycles. One pair of **elim_color** and **elim_cnt** must be delivered in each cycle, in exact cascade order from level 1 to level n.
10. If **chain_num** = 0, **out_valid** must be high for exactly 1 cycle, and both **elim_color** and **elim_cnt** must be 0.

Specifications

1. Top module name: ZUMA (design file name: ZUMA.v).
2. It is an asynchronous reset and active-low architecture. If you use synchronous reset in your design, you may fail to reset the signals correctly.
3. The clock period must be less than **15 ns**.
4. The reset signal (**rst_n**) will be given only once at the beginning of simulation. All output signals should be reset. The pattern will check the output signals 100 ns after the reset signal is pulled low.
5. The signals **chain_num**, **elim_color**, and **elim_cnt** must be reset to 0 whenever **out_valid** is low. In addition, **elim_color** and **elim_cnt** must be 0 whenever **chain_num** is 0.
6. **out_valid** must not be high when either **in_valid** or **shot_valid** is high.
7. The latency for each shot is a maximum of 1000 cycles. Latency is the number of clock cycles

between the rising edge of `clk` when `shot_valid` is high and the falling edge of `out_valid` for that shot.

8. `chain_num`, `elim_color`, and `elim_cnt` must be correct whenever `out_valid` is high.
9. `out_valid` must be high for exactly `chain_num` continuous cycles or exactly 1 when `chain_num` == 0, and `chain_num` must remain constant throughout.
10. Multiple cascade outputs must be delivered in the **exact order** in which the eliminations occur: level 1 -> level 2 -> ... -> level n. The output order cannot be exchanged.
11. No output is allowed during the ring loading phase. `out_valid` must remain low whenever `in_valid` is high.
12. The input delay is set to $0.5 \times (\text{clock period})$.
13. The output delay is set to $0.5 \times (\text{clock period})$, and the output loading is set to **0.05**.
14. The total cell area is limited to **1,200,000**.
15. The synthesized design must **not** contain any **latches**.
16. Note that maximum synthesis time could not be over **20 minutes**.
17. Gate-level simulation **cannot** include any **timing violations** without the `notimingcheck` command.
18. After synthesis, the slack at the end of the timing report should be **non-negative (MET)**.
19. Do not use any wire / reg / submodule / variable name containing “**error**”, “**congratulation**”, “**latch**” or “**fail**”, or you will fail the lab.
20. No **ERROR** is allowed in simulation/synthesis log.
21. **Hardcoding answers** or using a **lookup table based** on the test case index is **prohibited**; your design must compute outputs correctly based on the actual inputs. You will get 0 point if you use this method, **no second chance**.

Grading Policy

1. **Function Validity:** 50% (Demo: Use your `ZUMA.v` with the `input.txt` and `PATTERN.v` we provide.) **1st demo and 2nd demo**.
2. **Pattern:** 30% (Demo: Use your `PATTERN.v` with the `input.txt` and `ZUMA_encrypted.v` we provide.) **1st demo only, NO 2nd demo for Pattern**.
 - (15%) Pattern correctness — Complete the simulation when the design is correct.
 - (2.5%) SPEC 4 — All outputs are set to 0 when reset.
 - (2.5%) SPEC 5 — Output values are set to 0 when `out_valid` is 0. `elim_color` and `elim_cnt` must be 0 whenever `chain_num` is 0.
 - (2.5%) SPEC 6 — `out_valid` should not overlap with `in_valid` or `shot_valid`.
 - (2.5%) SPEC 7 — Exceed maximum output latency.
 - (2.5%) SPEC 8 — Answer should be correct.

- (2.5%) SPEC 9 — **out_valid** must be high for exactly **chain_num** continuous cycles or exactly 1 when **chain_num** == 0, and **chain_num** must remain constant.

3. Performance: 20%

- Area × total execution latency × cycle period
- 1st demo and 2nd demo

- The grade for the 2nd demo would be **30% off the Functionality + Performance score**.
- The design we provide will also check the pattern correctness. If the simulation cannot complete because of an error in your pattern, the pattern demo will fail.

Check the result

Check Pattern

- Include **ZUMA_encrypted.v** in your **TESTBED.v**.
- Check each specification in terminal.

Specification	Command	Check the result on screen
Pattern correctness	<code>./01_run_vcs_rtl CORRECT</code>	Congratulations
SPEC 4	<code>./01_run_vcs_rtl SPEC_4_{1~4}</code>	SPEC-4 FAIL
SPEC 5	<code>./01_run_vcs_rtl SPEC_5_{1~5}</code>	SPEC-5 FAIL
SPEC 6	<code>./01_run_vcs_rtl SPEC_6_{1~2}</code>	SPEC-6 FAIL
SPEC 7	<code>./01_run_vcs_rtl SPEC_7_1</code>	SPEC-7 FAIL
SPEC 8	<code>./01_run_vcs_rtl SPEC_8_{1~10}</code>	SPEC-8 FAIL
SPEC 9	<code>./01_run_vcs_rtl SPEC_9_{1~2}</code>	SPEC-9 FAIL

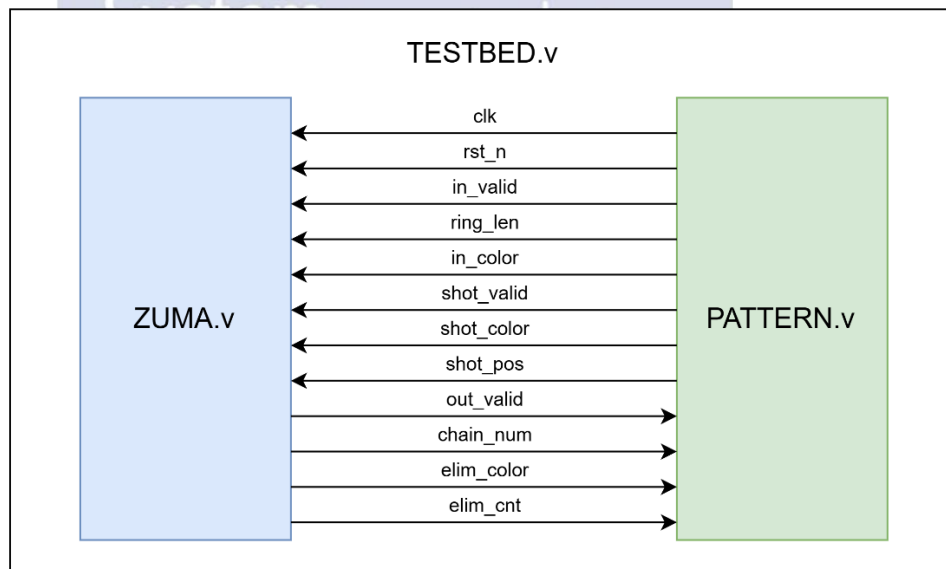
- You can run `./07_check_pattern` to check all above specifications.
- Make sure the keyword printed on the screen is EXACTLY THE SAME as the one listed in the table above. You can print out other information according to your preference, but only ONE keyword from the table above is permissible and it is indispensable.

Check Design

- Include **ZUMA.v** in your **TESTBED.v**.
- Check each specification in terminal.

Specification	Command	Check the result on screen
Pattern correctness	<code>./01_run_vcs_rtl</code>	Congratulations

Block Diagram



Note

- Please submit **PATTERN.v** and **ZUMA.v** in Lab03/EXERCISE/09_SUBMIT.
 - 1st demo deadline: **10/05 12:00 pm**
 - 2nd demo deadline: **10/07 12:00 pm**
- In this lab, you can adjust your clock cycle time. Consequently, make sure to key in your clock cycle time after the command like the figure below. It means that the TA will demo your design under this clock cycle time:

```
Exercise/09_SUBMIT]$ ./00_tar 15.0
```

- After that, you should check the following files under 09_SUBMIT/Lab03_iclabXXX/
 - ZUMA_iclabXXX.v (XXX is your account number)
 - PATTERN_iclabXXX.v
 - filelist_iclabXXX.f
 - clock_period_iclabXXX.txt (ex: 15.0_iclab999.txt)
 - If your file **violates the naming rule**, you will **lose 5 points**.
 - Encrypting any uploaded file is prohibited and will result in a failed demo.

If you miss any files on the list, you will fail this lab.

Use the command like the figure below to check if the files are uploaded or not.

```
Exercise/09_SUBMIT]% ./02_check 1st_demo
```

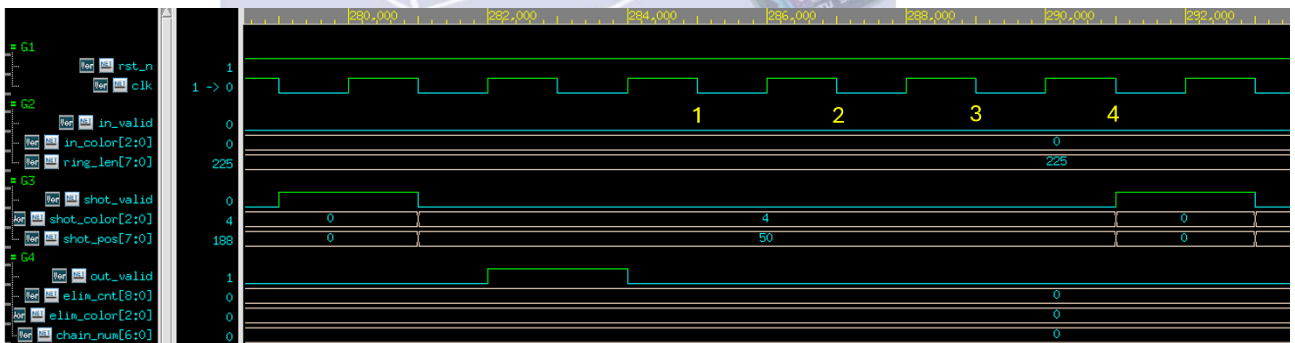
- We provide **ZUMA_encrypted.v**. You only need to complete the pattern and check the correctness of each design we provide.
- You need to implement **ZUMA.v**. We only check the functionality using our pattern.
- There will be no hidden design, but there **will be hidden pattern**.

Example Waveform

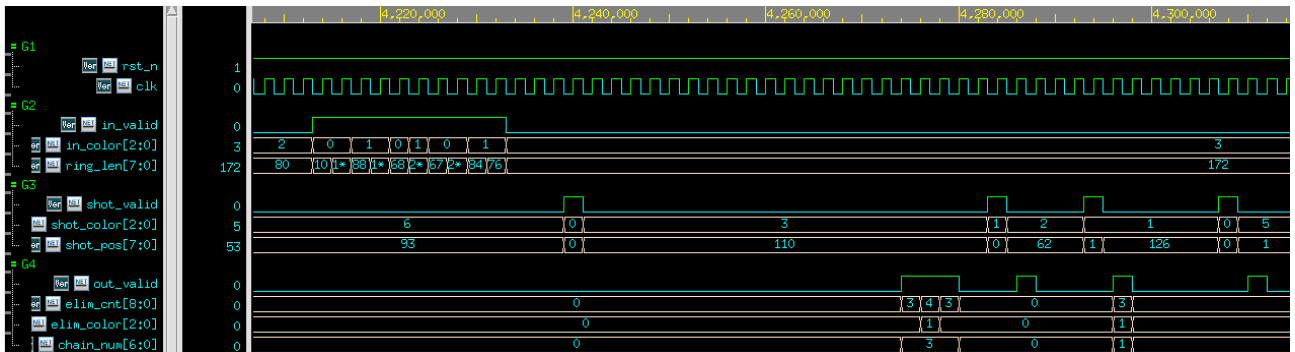
- Asynchronous reset and active-low reset all output to low.



- The next shot or next game starts only after the previous **out_valid** has returned low for the required 1~4 **negative-edge** interval.



- Input Output Waveform



- During ring loading, **in_valid** is continuously high for **ring_len** cycles, while **out_valid** stays low.
- Each valid shot is presented with a one-cycle **shot_valid** pulse.
- If **chain_num** = $n \geq 1$, **out_valid** stays high for exactly n continuous cycles, and one (**elim_color**, **elim_cnt**) pair is output per cycle in cascade order.
- If **chain_num** = 0, **out_valid** stays high for exactly one cycle with **elim_color** = 0 and **elim_cnt** = 0.